Laboratorio 2 – Haskell

Paradigmas de Programación

Estudiante

Felipe Alarcón Peralta

Profesor

Juan Camilo Ángel Hernández

Escuela Colombia de Ingeniería Julio Garavito

10/10/2026

Ejercicio 1 - Manejo de listas

1]:
```haskell
-- Usando recursión

miLength :: [a] -> Int
miLength [] = 0
miLength (x:xs) = 1 + miLength xs

miLast :: [a] -> a
miLast [x] = x
miLast (x:xs) = miLast xs

duplicar :: [Int] -> [Int]
duplicar [] = []
duplicar (x:xs) = (x*2) : duplicar xs

eliminarPares :: [Int] -> [Int]
eliminarPares [] = []
eliminarPares (x:xs)
    | mod x 2 == 0 = eliminarPares xs
    | otherwise = x : eliminarPares xs


-- Usando funciones de orden superior

miLength2 :: [a] -> Int
miLength2 xs = sum (map (\x -> 1) xs)

miLast2 :: [a] -> a
miLast2 [x] = x
miLast2 xs = head (reverse xs)

duplicar2 :: [Int] -> [Int]
duplicar2 xs = map (*2) xs

eliminarPares2 :: [Int] -> [Int]
eliminarPares2 xs = filter odd xs
```

Use foldr

Found:
```haskell
miLength [] = 0
miLength (x : xs) = 1 + miLength xs
```

Why Not:
```haskell
miLength xs = foldr (\ x -> (+) 1) 0 xs
```

Use map

Found:
```haskell
duplicar [] = []
duplicar (x : xs) = (x * 2) : duplicar xs
```

Why Not:
```haskell
duplicar xs = map (* 2) xs
```

Use even

Found:
```haskell
mod x 2 == 0even x
```

Why Not:

Ejercicio 2 - Listas por comprensión

•[5]:
```haskell
cuadrados :: [Int]
cuadrados = [x^2 | x <- [1..10]]

divisibles3 :: [Int]
divisibles3 = [x | x <- [1..30], mod x 3 == 0]

pares :: [(Int, Int)]
pares = [(x,y) | x <- [1..5], y <- [1..5], x < y]


- 2.

divisores :: Int -> [Int]
divisores n = [x | x <- [1..n], n > 0, mod n x == 0]


- 3.

divisoresPropios :: Int -> [Int]
divisoresPropios n = [x | x <- [1..n-1], n > 1, mod n x == 0]

esPerfecto :: Int -> Bool
esPerfecto n = n > 0 && sum (divisoresPropios n) == n

main :: IO ()
main = do
    print cuadrados
    print divisibles3
    print pares
    print (divisores 6)
    print (divisoresPropios 6)
    print (esPerfecto 6)
```

Move guards forward

Found:
```haskell
[x | x <- [1 .. n], n > 0, mod n x == 0][x | n > 0, x <- [1 .. n], mod n x == 0]
```

Why Not:

Move guards forward

Found:
```haskell
[x | x <- [1 .. n - 1], n > 1, mod n x == 0][x | n > 1, x <- [1 .. n - 1], mod n x == 0]
```

Why Not:

Ejercicio 3 - Funciones de orden superior

```haskell
import Data.Char (isUpper, toLower)

-- 1.

sumaLista :: [Int] -> Int
sumaLista xs = sum xs

mayoresQue10 :: [Int] -> Int
mayoresQue10 xs = length (filter (>10) xs)

productoLista :: [Int] -> Int
productoLista xs = product xs

celsiusAFahrenheit :: [Double] -> [Double]
celsiusAFahrenheit xs = map (\x -> x * 9 / 5 + 32) xs

ajustarImpares :: [Int] -> [Int]
ajustarImpares xs = map (\x -> if odd x then x + 5 else x) xs


-- 2.

nombresLargos :: [String] -> [String]
nombresLargos xs = filter (\x -> length x > 5) xs

soloNegativos :: [Int] -> [Int]
soloNegativos xs = filter (<0) xs

edadesUniversitarios :: [Int] -> [Int]
edadesUniversitarios xs = filter (\x -> x >= 18 && x <= 25) xs


-- 3.

agregarIVA :: [Double] -> [Double]
agregarIVA xs = map (\x -> if x >= 1000 then x * 1.19 else x) xs

longitudesVocales :: [String] -> [Int]
longitudesVocales xs = map length (filter empiezaVocal xs)
    where
        empiezaVocal [] = False
        empiezaVocal (x:xs) = toLower x `elem` "aeiou"

cuadradosPares :: [Int] -> [Int]
cuadradosPares xs = map (^2) (filter even xs)


-- 4.

hayNegativo :: [Int] -> Bool
hayNegativo xs = any (<0) xs

todosPares :: [Int] -> Bool
todosPares xs = all even xs

nombresCorrectos :: [String] -> Bool
nombresCorrectos xs = all empiezaMayuscula xs
    where
        empiezaMayuscula [] = False
        empiezaMayuscula (x:xs) = isUpper x


-- 5. Funciones usando takeWhile o dropWhile

menoresQue100 :: [Int] -> [Int]
menoresQue100 xs = takeWhile (<100) xs

sinCerosIniciales :: [Int] -> [Int]
sinCerosIniciales xs = dropWhile (==0) xs


-- Pruebas

main :: IO ()
main = do
    print (sumaLista [1,2,3,4])
    print (mayoresQue10 [5,12,15,8,20])
    print (productoLista [1,2,3,4])
    print (celsiusAFahrenheit [0,25,100])
```

Ejercicio 4 - Funciones que reciben y retornan funciones

```haskell
-- 1.
procesarLecturas :: (Double -> Double) -> [Double] -> [Double]
procesarLecturas f xs = map f xs

celsiusAFaranheit :: Double -> Double
celsiusAFaranheit c = c * 9 / 5 + 32

-- 2.
crearFiltro :: (a -> Bool) -> ([a] -> [a])
crearFiltro f xs = filter f xs

-- 3.
enRango :: Int -> Int -> (Int -> Bool)
enRango minimo maximo x = x >= minimo && x <= maximo

-- 4.
potenciador :: Int -> (Int -> Int)
potenciador n x = x ^ n

main :: IO ()
main = do
    print (procesarLecturas celsiusAFaranheit [0,12,20])

    let soloPares = crearFiltro even
    print (soloPares [1..10])

    let entre5y10 = enRango 5 10
    print (entre5y10 7)

    let potencia5 = potenciador 5
    print (potencia5 7)
```

Move guards forward

Found: **Why Not:**

```haskell
[x | x <- [1 .. n], n > 0, mod n x == 0][x | n > 0, x <- [1 .. n], mod n x == 0]
```

Move guards forward

Found: **Why Not:**

```haskell
[x | x <- [1 .. n - 1], n > 1, mod n x == 0][x | n > 1, x <- [1 .. n - 1], mod n x == 0]
```

Ejercicio 5 - “El problema de las dos jarras” usando A*

[34]:

```
import Data.List (sortOn)

type Estado = (Int, Int)
type Nodo   = Estado
type Costo  = Int
type Grafo  = [(Nodo, [(Nodo, Costo)])]

capA, capB :: Int
capA = 5
capB = 3

meta :: Estado
meta = (4, 0)

-- Verter de A a B
transferirAB :: Estado -> Estado
transferirAB (a, b) = (a - t, b + t)
  where t = min a (capB - b)

-- Verter de B a A
transferirBA :: Estado -> Estado
transferirBA (a, b) = (a + t, b - t)
  where t = min b (capA - a)

-- Sucesores
vecinos :: Estado -> [(Estado, Costo)]
vecinos (a, b) =
  [ ((capA, b), 1)
  , ((a, capB), 1)
  , ((0, b), 1)
  , ((a, 0), 1)
  , (transferirAB (a, b), 1)
  , (transferirBA (a, b), 1)
  ]

-- Heurística admisible
heuristica :: Estado -> Costo
heuristica e@(a, b)
  | e == meta  = 0
  | a == 4     = 1
  | a + b == 4 = 1
  | otherwise  = 2

-- A* base (devuelve nodos expandidos en orden)
astar :: (Nodo -> [(Nodo, Costo)]) -> (Nodo -> Costo) -> Nodo -> Nodo -> [(Nodo, Costo)]
astar sucesores h inicio objetivo = buscar [(inicio, 0)] []
  where
    buscar [] _ = []
    buscar ((nodo, costo):cola) visitados
      | nodo == objetivo = [(nodo, costo)]
      | nodo `elem` visitados = buscar cola visitados
      | otherwise =
          let nuevos = [ (v, costo + c) | (v, c) <- sucesores nodo, v `notElem` visitados ]
              ordenados = sortOn (\(v, c) -> c + h v) (cola ++ nuevos)
          in (nodo, costo) : buscar ordenados (nodo : visitados)

-- A* que devuelve el camino solución
astarCamino :: (Nodo -> [(Nodo, Costo)]) -> (Nodo -> Costo) -> Nodo -> Nodo -> Maybe ([Nodo], Costo)
astarCamino sucesores h inicio objetivo = buscar [(inicio, 0, [inicio])] []
  where
    buscar [] _ = Nothing
    buscar ((n, c, cam):cola) vis
      | n == objetivo = Just (reverse cam, c)
      | n `elem` vis  = buscar cola vis
      | otherwise =
          let nuevos = [ (v, c + k, v:cam) | (v, k) <- sucesores n, v `notElem` vis ]
              ord = sortOn (\(v, g, _) -> g + h v) (cola ++ nuevos)
          in buscar ord (n : vis)

main :: IO ()
main = do
  putStrLn "Nodos expandidos (A* base):"
  print (astar vecinos heuristica (0,0) meta)
  putStrLn "\nCamino solución:"
  print (astarCamino vecinos heuristica (0,0) meta)

main
```

```
Nodos expandidos (A* base):
[((0,0),0),((5,0),1),((0,3),1),((5,3),2),((2,3),2),((3,0),2),((2,0),4),((4,3),4),((0,2),4),((5,1),4),((5,2),5),((0,1),5),((4,3),6),((4,0),7)]

Camino solución:
Just ([(0,0),(5,0),(2,3),(2,0),(0,2),(5,2),(5,0),(4,1),(4,0)],7)
```

Retrospectiva

1. **¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas/Hombre)**

   Invertí aproximadamente 10 horas en el desarrollo del laboratorio divididas en 5 dias, incluyendo el análisis de los ejercicios, la escritura del código, las pruebas y la corrección de errores

2. **¿Cuál es el estado actual del laboratorio? ¿Por qué?**

   El laboratorio se encuentra parcialmente completado. Logré desarrollar varias funciones y comprender mejor conceptos como las listas por comprensión, la recursión y las funciones de orden superior. Sin embargo, tuve dificultades con la implementación del algoritmo A* en el problema de las dos jarras, por lo que todavía necesito revisar y comprobar su funcionamiento.

3. **¿Cuál consideran fue el mayor logro? ¿Por qué?**

   Considero que el mayor logro fue completar la mayoría de los ejercicios y entender cómo se pueden resolver problemas utilizando diferentes herramientas de Haskell.

4. **¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?**

   El mayor problema técnico fue implementar y ejecutar correctamente el algoritmo A* para resolver el problema de las dos jarras. Tuve dificultades para manejar los estados

5. **¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?**

   Realicé el laboratorio de manera individual, por lo que tuve que organizar mi tiempo, investigar los temas y buscar soluciones por mi cuenta. Considero que hice bien en mantener la disposición para aprender y seguir intentando resolver los ejercicios a pesar de las dificultades con la programación en Haskell

6. **¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.**

- Anthropic. (s. f.). Claude. https://claude.ai/
- Google. (s. f.). NotebookLM. https://notebooklm.google.com/
- OpenAI. (s. f.). ChatGPT. https://chatgpt.com/